



LAMELLAR: A Rust-based asynchronous tasking and PGAS runtime for HPC

Nov 2024

Ryan Frieese,
Roberto Gioiosa, Joseph Cottam,
Erdal Mutlu, Greg Roek,
Polykarpos Thomadakis, Mark Raugas

Pacific Northwest National Lab

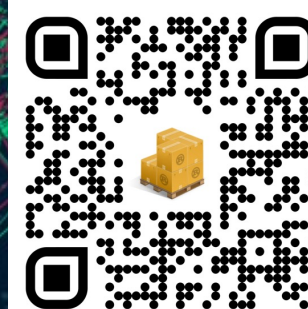


ryan.frieese@pnnl.gov
509-375-2903

PNNL is operated by Battelle for the U.S. Department of Energy



[https://github.com/
pnnl/lamellar-
runtime](https://github.com/pnnl/lamellar-runtime)



[https://crates.io/cr
ates/lamellar](https://crates.io/crates/lamellar)



[https://docs.rs/lamel
lar/latest/lamellar/](https://docs.rs/lamellar/latest/lamellar/)

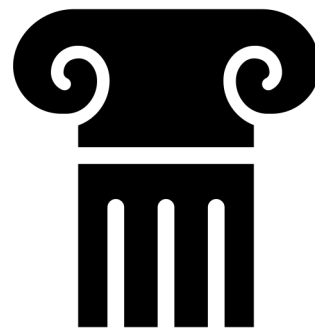
Why Rust?

Rust is a modern systems programming language that is an alternative to C and C++. It prioritizes reliability, performance, and productivity.



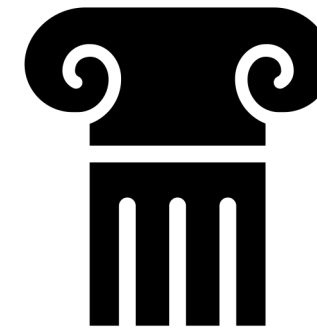
Reliability

- Memory and Thread safety
- Strong compile-time guarantees



Performance

- No garbage collector or runtime
- Zero-cost abstractions



Productivity

- Cargo build tool and package manager
- Verbose and helpful compiler messages



Community Support

- Mozilla 2006 – V1.0 2015 – V1.61.0 May'22
- U.S. Government recommends switching to memory safe language
- Industry Adoption: Amazon, Dropbox, Facebook, Google, Microsoft, etc.
 - The Rust Foundation
 - Microsoft & Google: 70% of vulnerabilities are memory related
- Voted “most loved programming language” on stack overflow 2016-2023

LAMELLAR Stack & Nomenclature

Lamellar

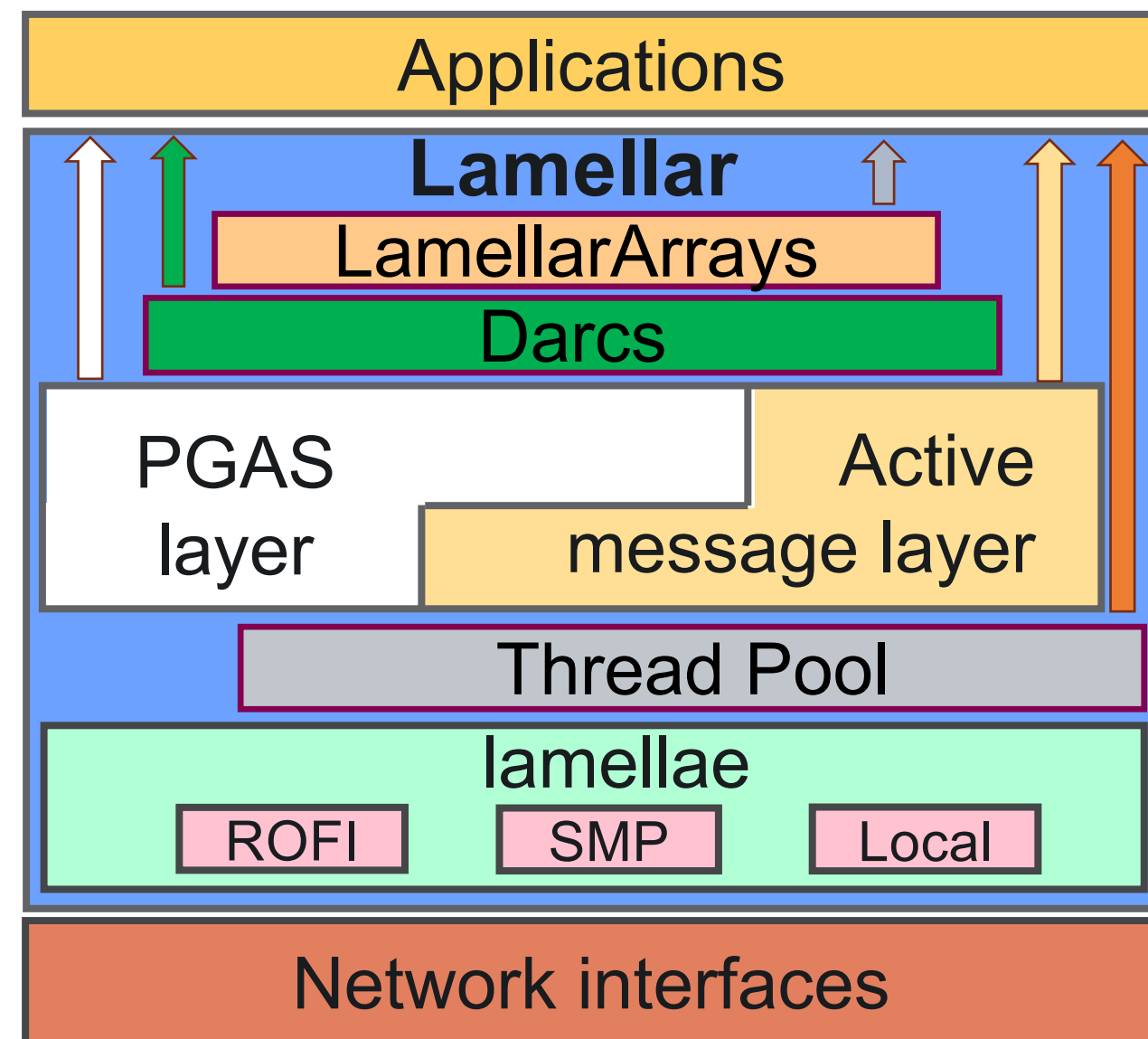
- A distributed, asynchronous tasking and PGAS runtime for HPC
- **Active Messages:** Distributed async tasks
- **Darcs:** Distributed Arcs (Atomic Reference Counted pointers)
- **LamellarArrays:** High(er)-level data structure

Lamellae

- Interface between network devices/drivers and upper-level runtime layers
- Support High Performance Infiniband networks

PE – Processing Element ($\cong$ MPI Rank)

- Supports multiple OS threads
- Multiple PEs per hardware node (e.g., one per socket)
- **Thread pool** for Active Messages and Futures



LAMELLAR: Active Messages (AM)

- Asynchronous remote execution (async/await)
- Implemented via Procedural Macros
 - topic for another time
- `#[lamellar::AmData]` – Extends `#[Derive(...)]`
 - Applied to user defined structs
 - Represents AM Input data
- `#[lamellar::am]` – Attribute proc macro
 - Applied to the user defined impl of the **LamellarAM** trait
 - Represents an AM task
- Can return data or a “callback” AM to the original PE
- Point to point (e.g. exec on pe p)
- Collective (e.g. exec on all)

Active Message Data + Processing Instructions

e.g. pointer to function that takes data as input

Replaces `#[Derive(...)]`

The data

```
#[AmData(Debug)]
struct HelloWorldAm {
    msg: String
    orig_pe: usize
}
```

Attribute macro

```
#[am]
impl LamellarAm for HelloWorldAm {
    async fn exec(self) {
        Println!("{:?}", self);
    }
}
```

The processing instructions

Execute on a specific PE

```
//main()...
world.exec_am_pe(pe,
    HelloWorldAm {
        msg: "Hello World",
        orig_pe: world.my_pe()
    }
);
```

An AM instance

LAMELLAR: Darc<T> - Distributed Atomic Reference Count

Macro implements specialized (de)serialization methods to appropriately track reference counts when sending to remote PEs

```
#[lamellar::AmData(Clone)]  
struct DarcAm{  
    darc: Darc<AtomicUsize>,  
}
```

Each PE contains a local instance of the interior object (in this example, not directly mutable with RDMA)

```
#[lamellar::am]  
impl LamellarAm for DarcAm{  
    fn exec(&self) {  
        self.darc.fetch_add(1, Ordering::Relaxed);  
    }  
}
```

Mutability controlled via atomic

```
fn main(){  
    let team = world.create_team...;  
    let am = DarcAm {darc: Darc::new(&team, AtomicUsize::new(0))}  
    for i in 0..1000 { team.exec_am_pe(i%num_pes, am.clone()) }  
}
```

Allocated on either the world or a team

- Similar to an Arc but in distributed memory
 - Global lifetime – value is valid (and accessible) if *any* PE has a reference
 - Inner mutability **disallowed** by default
- Intended for use in active messages
 - Enables distributed access and manipulation of generic Rust objects

LAMELLAR LamellarArray Types

	Type	Protection	Notes
Atomic	ReadOnlyArray	Not needed	
	NativeArray	Element-wise	Language-based atomics
	GenericArray		Mutex-based atomics
Lock	Local	PE-wise	
	Global	Cluster-wise	
	UnsafeArray	None	For Internal use (requires 'unsafe' blocks)

- Global or local indexing
- Iterators for idiomatic Rust
- Sub-arrays supported
- Block- or cyclic-data distribution
- Operation order over indices **not** guaranteed
- Safely converted at runtime. (Memory regions will have only one array type at any time.)
- Built on Darcs

Iterator Type	Data Parallel	Data Transfer	Executing PEs
Distributed	Yes	Maybe	All
Local	Yes	No	All
Serial	No	Yes	Calling

LAMELLAR: LamellarArray Example

Set of pes the data should exist on

Global length

Currently also support "Cyclic"

```
fn main(){
  //initialize lamellar
  let ll_array = LocalLockArray::<usize>::new(team, ARRAY_LEN, Distribution::Block);
  ll_array.dist_iter_mut()
```

Composable
iteration
functions

```
[.enumerate()
 .filter(|(i,_)| i % 2 == 0)
 .for_each(|(i,e)| *elem = i*(my_pe)).block();
```

Mutable data parallel iteration
(safety managed by array type)

Block thread until
distributed op is completed

```
let ro_array = gl_array.into_read_only();
let idx = vec![1,100,32,54];b
let vals_handle = ro_array.batch_load(idx).spawn();
//do stuff
```

spawn distributed op
to worker thread

Supports single and batched
element-wise operations

Each array "into" call is collective
and blocks until there is only a single
reference to the array on each PE

```
let vals = vals_handle.block();
let atomic_array = ro_array.into_atomic();
atomic_array.add(idx[0],val[0]).block();
let sum = atomic_array.sum().block();
}
```

It is not possible to have two
different Array Types referencing the
same underlying data

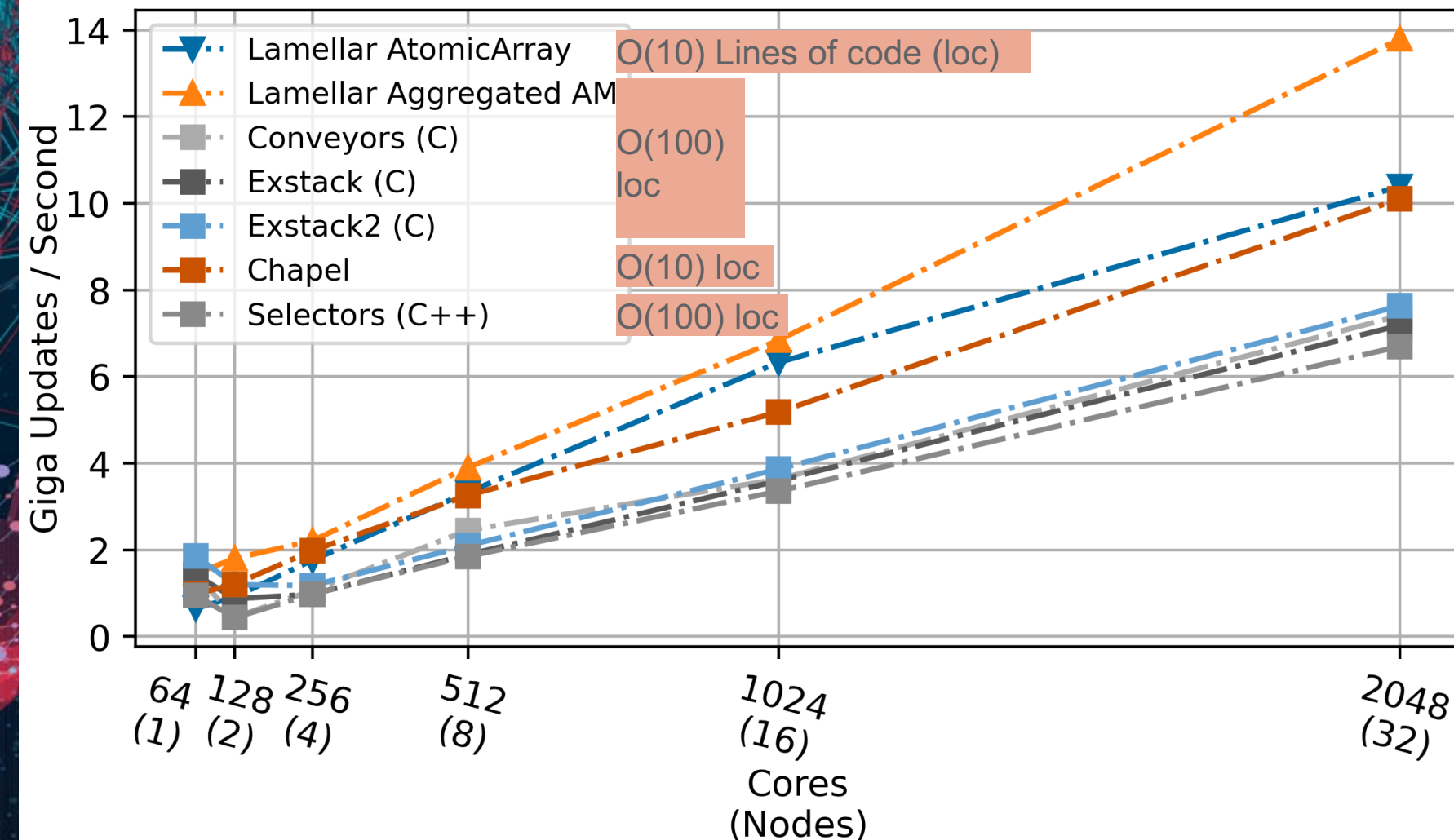
Supports common and user-defined reductions

Experiments/Benchmarks

- Results presented were performed on a PNNL Cluster
 - 48 node (32 Nodes available)
 - 2 socket – AMD EPYC 7543 32-Core
 - ✓ L1 - 64K
 - ✓ L2 – 512KB
 - ✓ L3 – 32MB
 - ✓ 8 Numa nodes per socket
 - 256 GB memory per node
 - Network: Mellanox HDR-100 ConnectX-6 InfiniBand HBA
 - ✓ Peak bandwidth of 100Gb/s (12.5GB/s)
 - ✓ Full fat tree
- Bale/Conveyors/Selectors built using OpenSHMEM from OpenMPI 5.0.0rc2
- Chapel v2.0

Histogram

Random increments on a distributed array



- AtomicArray
 - Element-wise atomic adds
 - Runtime provided aggregation
 - `array.batch_add(indicies,1);`
- Aggregated Am
 - User implemented AM aggregation

Configuration:

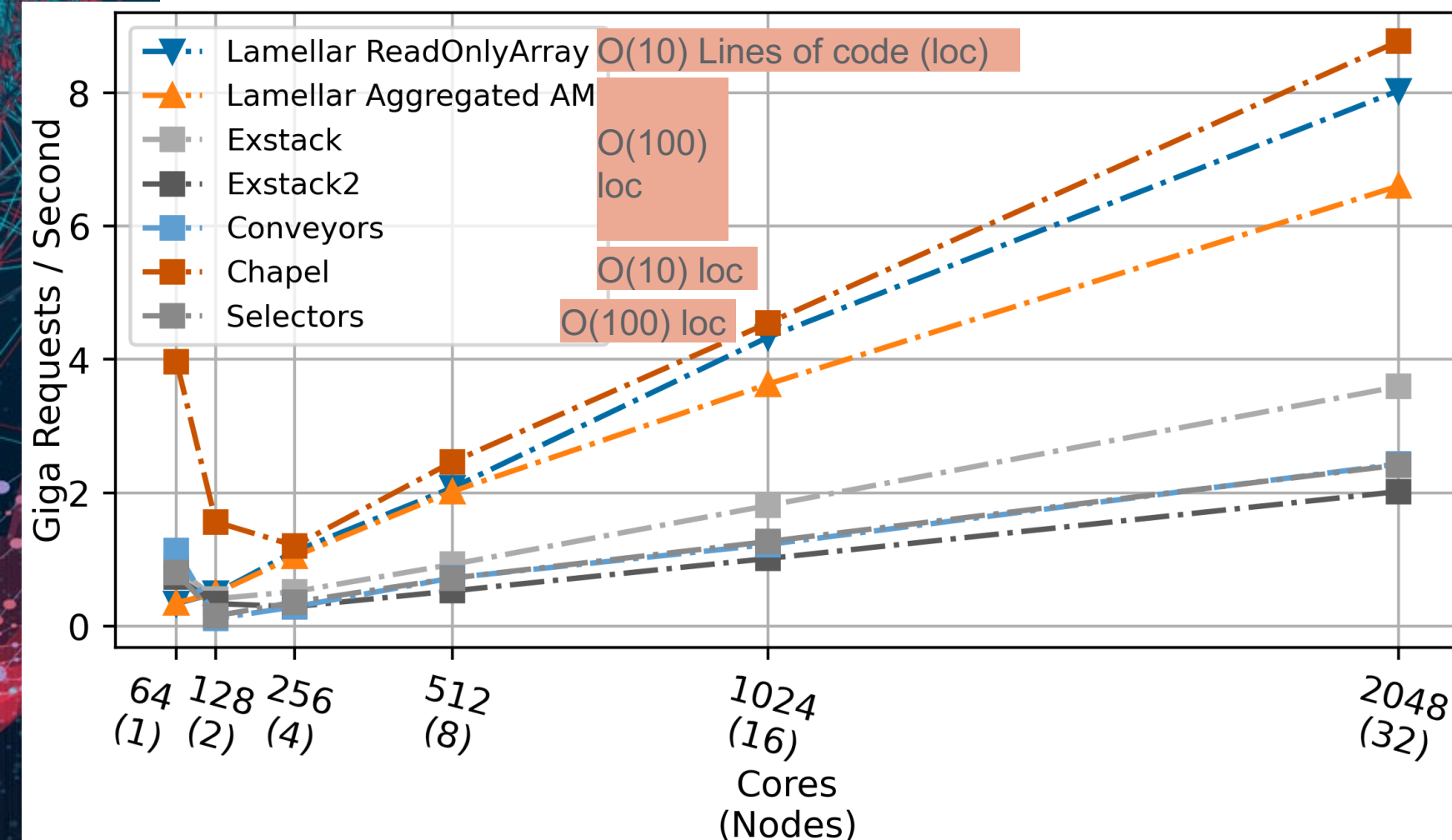
- 1000 elements per core
- 10 Million Updates per Core
- 10 runs
- 10,000 indices per "batch"
- Lamellar: 16 Pes per Node, 4 threads per PE
- Chapel: (1-8) Locales(PE) per Node
- Exstack, Exstack2, Conveyors, Selectors: 1 PE per thread

```
for idx in random_indices {
    array[idx] += 1
}
```

One-way
communication

IndexGather

Randomly gather elements in
a distributed array



• ReadOnlyArray

- Element-wise loads
- Runtime provided aggregation
- `array.batch_load(indicies) -> Vec<usize>;`

• Aggregated AM

- User implemented AM aggregation

Configuration:

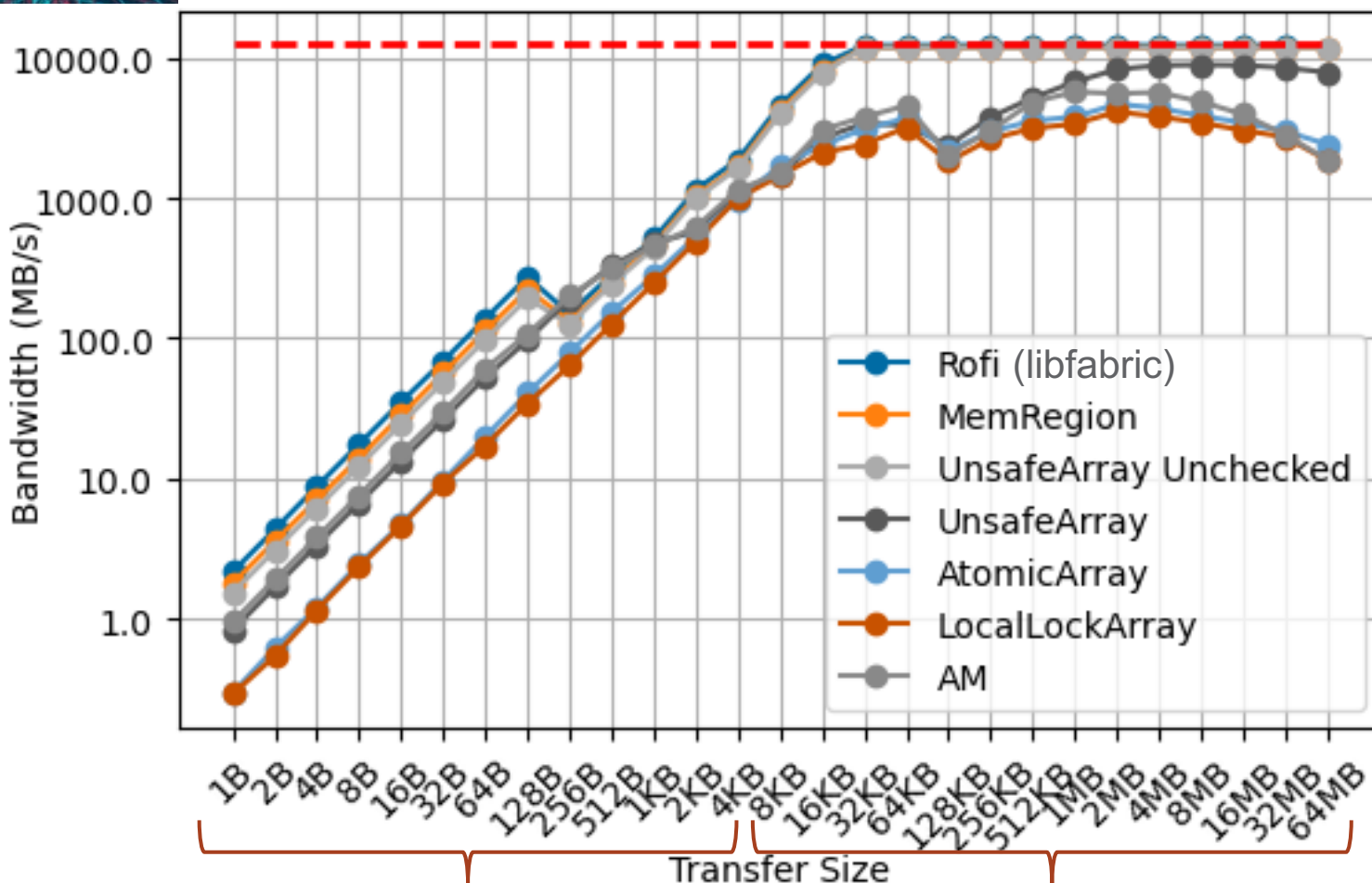
- 1000 elements per core
- 10 Million Updates per Core
- 10,000 indices per “batch”
- 10 runs
- Lamellar: 16 Pes per Node, 4 threads per PE
- Chapel: (1-8) Locales(PE) per Node
- Exstack, Exstack2, Conveyors, Selectors: 1 PE per thread

```
for (i,idx) in random_indices.enumerate() {
    result[i] = array[idx]
}
```

Two-way
communication

“Put” and “Get” Bandwidths

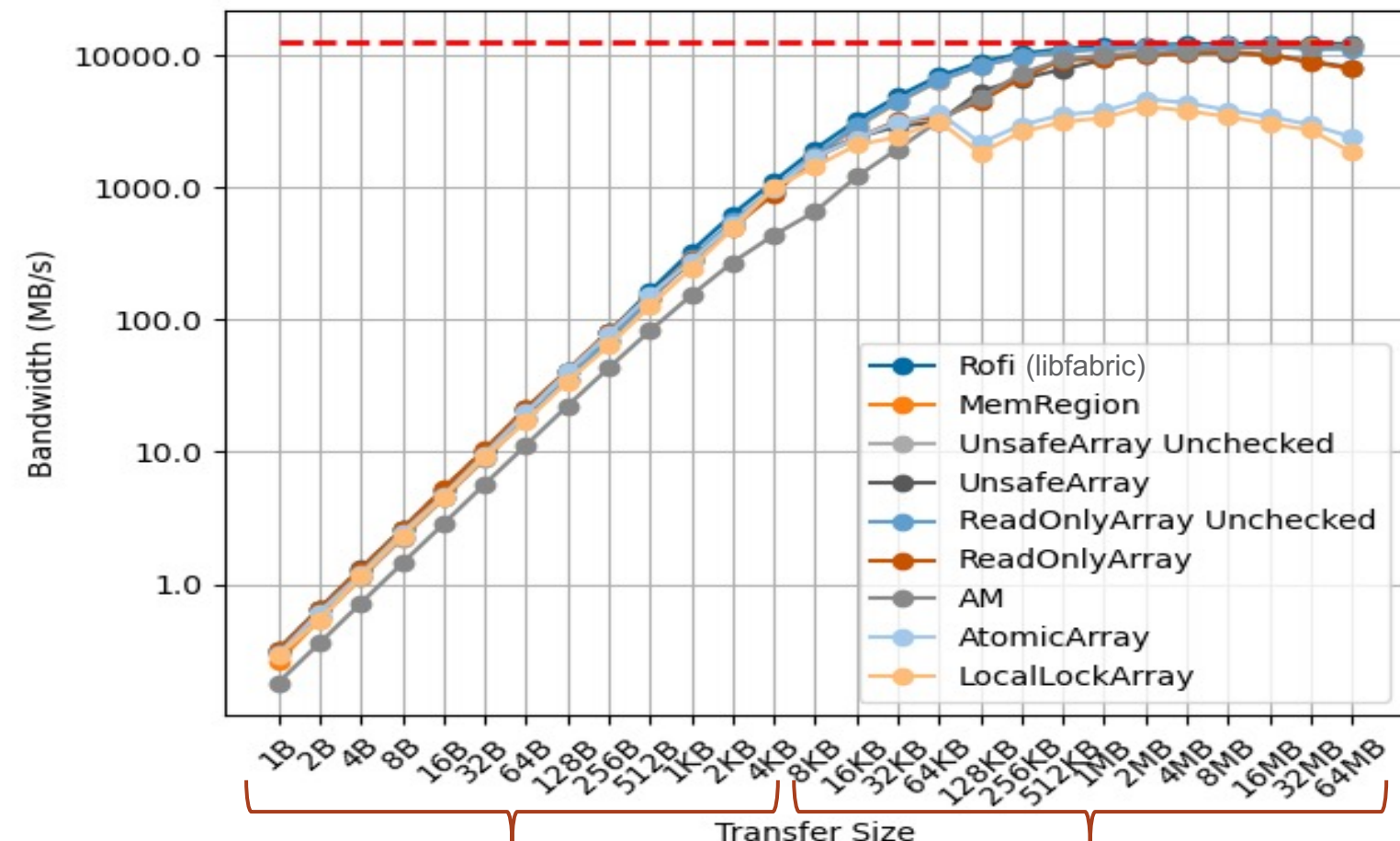
Puts



262143 requests

1GiB/size requests

Gets



262143 requests

1GiB/size requests

- Generally Unsafe Methods get better raw performance (but no guarantees on correctness) – currently working on supporting network accelerated atomic operations

LAMELLAR: Code availability

Our current stable release is V0.7

Our code is open source and available on GitHub under BSD license:

- Lamellar Runtime - <https://github.com/pnnl/lamellar-runtime>
- Lamellar Benchmarks - <https://github.com/pnnl/lamellar-benchmarks>



We also release the Runtime on Crates.io

- <https://crates.io/crates/lamellar>



Documentation with over 500 runnable examples

- <https://docs.rs/lamellar/latest/lamellar/>

